

Experiment No. 4

Title

"Write a C++ program to implement the matrix ADT using a class. The operations supported by this ADT are:

- a. Reading a matrix.
- b. Addition of matrices.
- c. Printing a matrix.
- d. Subtraction of matrices.
- e. Multiplication of matrices"

1. Aim

To study and implement the Matrix Abstract Data Type (ADT) using a C++ class by encapsulating matrix data and performing fundamental matrix operations such as reading, displaying, addition, subtraction, and multiplication using object-oriented programming concepts.

2. Learning Outcomes

After completing this experiment, students will be able to:

1. Design and implement a **Matrix ADT** using a C++ class.
2. Apply **object-oriented programming** concepts to represent matrix data and operations.
3. Use **constructors** for proper initialization of matrix objects.
4. Perform matrix operations such as **addition, subtraction, and multiplication**.
5. Read and display matrices using member functions.
6. Develop reusable and modular C++ programs using classes.
7. Analyze the conditions required for valid matrix operations.
8. Demonstrate problem-solving skills by implementing matrix manipulation algorithms using OOP concepts.

Course Outcomes mapped: CO1, CO2

3. Prerequisites

Students should have knowledge of:

- Basic C++ programming syntax
- Classes and Objects
- Arrays and Two-Dimensional Arrays
- Member Functions
- Constructors
- Basic Matrix concepts (rows, columns, and matrix operations)

4. Software / Tools Required

1. **Operating System:** Ubuntu Linux

2. **Programming Language:** C++
3. **Compiler:** GNU G++ (GCC)
4. **Text Editor:** Gedit or Nano or touch
5. **Terminal:** Ubuntu Terminal

5. Theory

A Matrix is a rectangular arrangement of elements organized into rows and columns. Matrices are widely used in mathematics, computer graphics, image processing, scientific computing, machine learning, and engineering applications. In programming, a matrix is generally represented as a two-dimensional array. However, using a class to implement a Matrix provides better data encapsulation, modularity, and code reusability by grouping the matrix data and related operations together.

A Matrix Abstract Data Type (ADT) defines a logical model for representing matrices and specifies the operations that can be performed on them without exposing the implementation details. In C++, the Matrix ADT can be implemented using a class where the data members store the matrix elements, number of rows, and number of columns, while member functions perform operations such as reading, displaying, addition, subtraction, and multiplication. This approach follows the principles of Object-Oriented Programming (OOP) by encapsulating both data and functions within a single unit.

5.1 Matrix Operations & Mathematical Formulas

Matrix Addition

Matrix addition is possible only when both matrices have the same order.

If A and B are two matrices of order $m \times n$, then:

$$C = A + B$$

$$C[i][j] = A[i][j] + B[i][j]$$

C++ syntax:

```
sum = m1.add(m2);
```

Matrix Subtraction

Matrix subtraction is also possible only when both matrices have the same order.

If A and B are two matrices of order $m \times n$, then:

$$C = A - B$$
$$C[i][j] = A[i][j] - B[i][j]$$

C++ syntax:

```
diff = m1.subtract(m2);
```

Matrix Multiplication

Matrix multiplication is possible only when the number of columns of the first matrix is equal to the number of rows of the second matrix.

If A is of order $m \times n$ and B is of order $n \times p$, then the result matrix C will be of order $m \times p$.

Formula:

$$C[i][j] = A[i][0] * B[0][j] + A[i][1] * B[1][j] + \dots + A[i][n-1] * B[n-1][j]$$

C++ syntax:

```
product = m1.multiply(m2);
```

6. Algorithm

The following steps outline the execution logic for implementing the Matrix Abstract Data Type (ADT) class using constructor initialization and operator overloading in C++:

Algorithm Description

Step 1: Start the program.

Step 2: Define a class named `Matrix`.

Step 3: Declare a 2D array to store matrix elements.

Step 4: Declare variables `rows` and `cols` to store the number of rows and columns.

Step 5: Define a function `read()` to input the number of rows, columns, and matrix elements.

Step 6: Define a function `print()` to display the matrix elements.

Step 7: Define a function `add()` to add two matrices element by element.

Step 8: Define a function `subtract()` to subtract two matrices element by element.

Step 9: Define a function `multiply()` to multiply two matrices using row-column multiplication.

Step 10: In the `main()` function, create objects for two input matrices and result matrices.

Step 11: Read the first matrix using the `read()` function.

Step 12: Read the second matrix using the `read()` function.

Step 13: Add the two matrices and store the result in another matrix object.

Step 14: Subtract the second matrix from the first matrix and store the result.

Step 15: Multiply the two matrices and store the result.

Step 16: Display the first matrix.

Step 17: Display the second matrix.

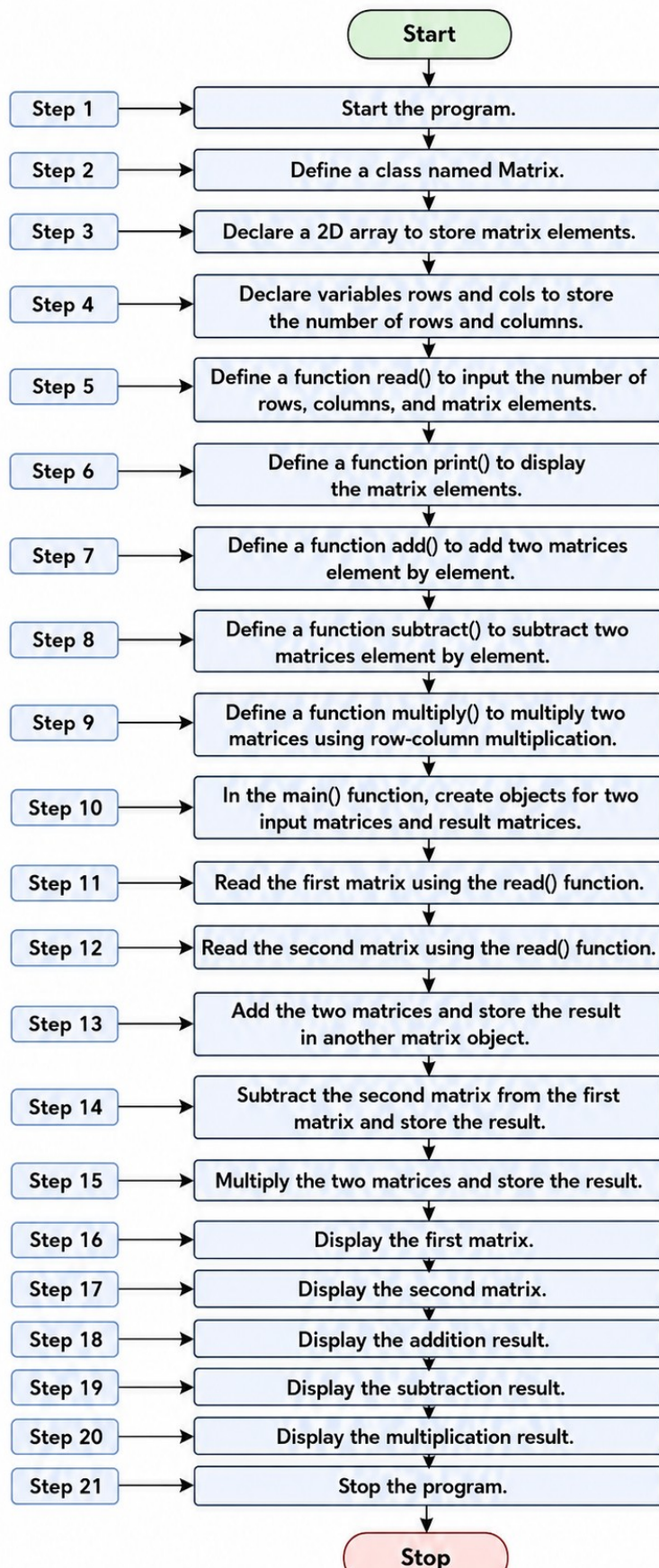
Step 18: Display the addition result.

Step 19: Display the subtraction result.

Step 20: Display the multiplication result.

Step 21: Stop the program.

The flowchart shown below illustrates the overall execution flow of the program.



7. Lab Exercise

Complete the following tasks sequentially to implement the required functionality of the Matrix.

Exercise 1: Define the `Matrix` class boundaries with its row, column, and storage configurations.

Exercise 2: Construct proper bounds checking inside the overloaded operators to handle error scenarios (e.g., trying to add matrices of mismatched sizes).

Exercise 3: Implement the core multiplication logic algorithm accurately.

Exercise 4: Compile and execute the code utilizing the g++ compiler environment.

8. Program

**** Develop a C++ program to to implement the matrix ADT using a class. The operations supported by this ADT are:**

- a. Reading a matrix.
- b. Addition of matrices.
- c. Printing a matrix.
- d. Subtraction of matrices.
- e. Multiplication of matrices

9. Sample Input

****Write output of Program.**

10. Observation

The overloaded `>>` and `<<` operators make passing matrix inputs and displaying outputs elegant, handling variable dimensions cleanly.

The system accurately blocks arithmetic execution and surfaces logical errors if dimensions fail mathematical compatibility rules.

The multi-layered loops in the `*` operator correctly aggregate product values according to classic matrix dot product mechanics.

11. Result

The Matrix ADT class was successfully developed using constructor initialization and operator overloading principles. Operations mapping to matrix initialization, console rendering, addition, subtraction, and multiplication execution run smoothly in compliance with standard matrix constraints.

13. Conclusion

Through this lab session, object-oriented custom math classes were contextualized using custom array collections. Overloading traditional operation markers like +, -, and * transforms typical modular method calls into intuitive design architecture patterns, laying strong optimization frameworks for real-world mathematical applications.

14. References

1. E. Balagurusamy, *Object-Oriented Programming with C++*, McGraw Hill.
2. Robert Lafore, *Object-Oriented Programming in C++*, Pearson.
3. Bjarne Stroustrup, *The C++ Programming Language*, Addison-Wesley.

15. FAQs

1. What is ADT?
2. What is Matrix ADT?
3. Why is a class used to implement Matrix ADT?
4. What data members are required for Matrix ADT?
5. What are the main functions implemented in Matrix ADT?
6. What is the condition for matrix addition?
7. What is the condition for matrix subtraction?
8. What is the condition for matrix multiplication?
9. What is the formula for matrix multiplication?
10. What is encapsulation?

Program Code Practical No-4

```
#include <iostream>

using namespace std;

class Matrix {
    int a[10][10];
    int rows, cols;

public:
    void read() {
        cout << "Enter number of rows and columns: ";
        cin >> rows >> cols;

        cout << "Enter matrix elements:\n";
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                cin >> a[i][j];
            }
        }
    }

    void print() {
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                cout << a[i][j] << " ";
            }
            cout << endl;
        }
    }

    Matrix add(Matrix m) {
        Matrix temp;
```

```
temp.rows = rows;
temp.cols = cols;

for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        temp.a[i][j] = a[i][j] + m.a[i][j];
    }
}

return temp;
}
```

```
Matrix subtract(Matrix m) {
    Matrix temp;
    temp.rows = rows;
    temp.cols = cols;

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            temp.a[i][j] = a[i][j] - m.a[i][j];
        }
    }

    return temp;
}
```

```
Matrix multiply(Matrix m) {
    Matrix temp;
    temp.rows = rows;
    temp.cols = m.cols;

    for (int i = 0; i < temp.rows; i++) {
```

```
        for (int j = 0; j < temp.cols; j++) {
            temp.a[i][j] = 0;
            for (int k = 0; k < cols; k++) {
                temp.a[i][j] += a[i][k] * m.a[k][j];
            }
        }
    }

    return temp;
}
};
```

```
int main() {
    Matrix m1, m2, sum, diff, product;

    cout << "Enter first matrix:\n";
    m1.read();

    cout << "Enter second matrix:\n";
    m2.read();

    sum = m1.add(m2);
    diff = m1.subtract(m2);
    product = m1.multiply(m2);

    cout << "\nFirst Matrix:\n";
    m1.print();

    cout << "\nSecond Matrix:\n";
    m2.print();

    cout << "\nAddition:\n";
```

```
sum.print();

cout << "\nSubtraction:\n";
diff.print();

cout << "\nMultiplication:\n";
product.print();

return 0;
}
```